

Data mining & machine learning

4. Regression

Francesco Pistolessi

University of Pisa

`francesco.pistolessi@unipi.it`

M.Sc. in Artificial Intelligence and Data Engineering



UNIVERSITÀ DI PISA

By the end of this lecture, you will be able to:

- perform an **exploratory data analysis** for a regression task;
- define and compute the main **error metrics**: R^2 , MAE, MSE, RMSE, MAPE;
- explain and compare **linear**, **Ridge** and **Lasso** regression;
- tune α in Lasso via **GridSearchCV** and connect it to model optimization;
- interpret **prediction error plots**;
- apply **cross-validation** for model comparison, avoiding data leakage.

Main topics

the dataset & EDA

error metrics

linear models

GridSearchCV + Lasso

model comparison

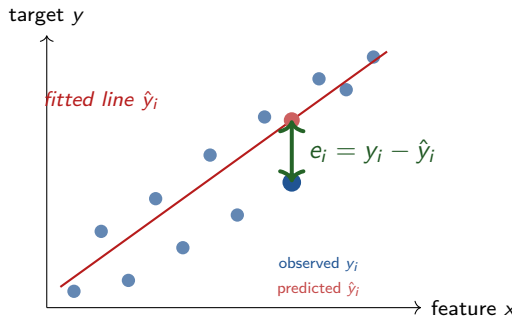
What is regression?

Regression is a supervised learning task whose output is a **continuous numerical value**.

The model learns a mapping $f : \mathbb{R}^P \rightarrow \mathbb{R}$. The quality of predictions is measured by how far $\hat{y}$ is from y , using distance-based metrics.

Real-world examples

- House price from size and location
- Disease progression from clinical measures
- Stock return from market features
- Energy demand from weather data



$e_i = y_i - \hat{y}_i$: the **residual** is the vertical gap between the true value (blue) and the model's prediction on the fitted line (red).

Example: the diabetes dataset

A classic benchmark from Efron et al. (2004), available in scikit-learn.

442 patients, 10 features, 1 continuous target.

Features (normalized: mean 0, std 1)

- age, sex, bmi (body mass index)
- bp (average blood pressure)
- s1 total cholesterol s2 LDL
- s3 HDL s4 TCH
- s5 log(triglycerides) s6 blood sugar

Target: quantitative measure of **disease progression** one year after baseline (continuous, range ≈ 25 –346).

```
from sklearn import datasets
import pandas as pd

data = datasets.load_diabetes()
df = pd.DataFrame(
    data.data,
    columns=data.feature_names)
df['target'] = data.target

df.describe().T
# mean    std    min    max
# age:    0.00   0.048   ...
# bmi:    0.00   0.047   ...
# ...
# tgt:   152.1  77.09   25   346
```

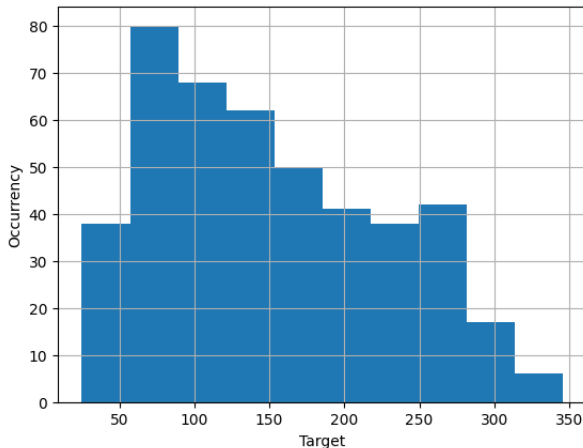
EDA: target distribution

```
df['target'].hist()  
plt.xlabel('Target')  
plt.ylabel('Occurrence')  
plt.show()
```

The target spans [25, 346] and is quite bell-shaped (slight right skew).

Why check the target first?

- Understand the prediction scale
- Detect extreme values that may distort metrics such as MSE
- Decide whether a transformation (e.g. log) might help



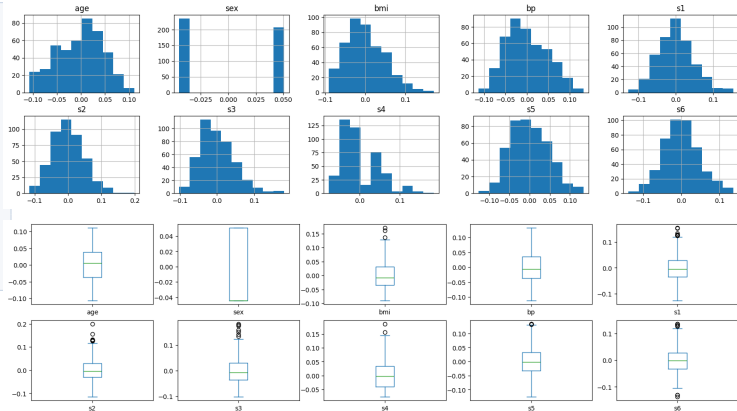
EDA: feature histograms and box plots

```
# Histograms: 2 rows x 5 cols
df[data.feature_names].hist(
    bins=10,
    layout=(2, 5),
    figsize=(20, 5))
```

```
# Box plots: 2 rows x 5 cols
df[data.feature_names].plot(
    kind='box',
    subplots=True,
    layout=(2, 5),
    figsize=(20, 5))
```

Key observations:

- sex is binary $\{-1, +1\}$
- s1–s6 are near-Gaussian after normalization
- bmi, bp, s5 show mild right skew
- Box plots confirm few outliers



EDA: correlation heatmap

```
sns.heatmap(df.corr(),
             annot=True, vmin=-1, vmax=1,
             cmap="coolwarm")
```

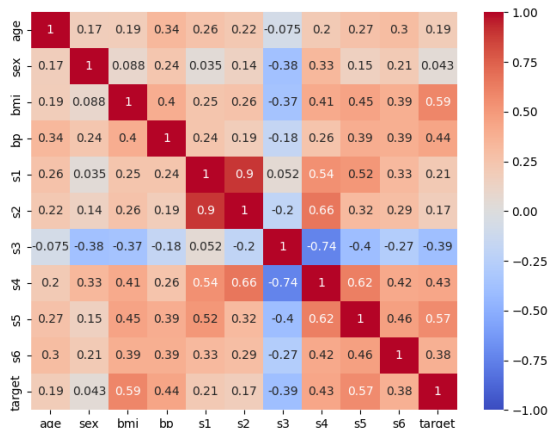
Correlations with target:

- Strong positive: bmi (0.59), s5 (0.57), bp (0.44)
- Negative: s3/HDL (−0.40)

Between features:

- s1 and s2 are highly correlated (0.90): they carry **redundant information**. Including both does not help and may hurt linear models.
- s3 and s4 are anti-correlated (−0.74).

Highly correlated features are a problem for linear models (they inflate coefficient variance). Tree-based models are more robust to this.



Error metrics: measuring prediction quality

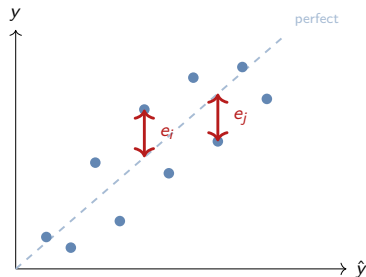
Let y_i be the **true value** and $\hat{y}_i$ the **predicted value** for sample i , over n total samples. The **residual** is $e_i = y_i - \hat{y}_i$.

A good metric should be:

- Zero for a **perfect** predictor
- Larger when predictions are **farther** from truth
- **Interpretable** and aligned with the task goal

Scale-dependent vs. unit-free

MAE, MSE, RMSE use the same units as y . They **cannot** be compared across tasks with different units. MAPE is **unit-free** and can be compared across tasks.

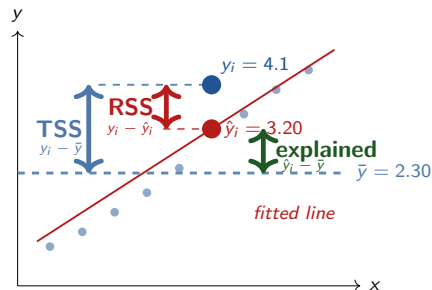


Each point is one sample. Points on the diagonal are perfect predictions. The farther a point from the diagonal, the larger the error.

R^2 : coefficient of determination

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- **TSS** (Total Sum of Squares): how much y varies around its mean $\bar{y}$ — the baseline difficulty of the task.
- **RSS** (Residual Sum of Squares): how much variance is still *unexplained* after fitting.
- $R^2 = 1 - \frac{\text{unexplained}}{\text{total}}$: **proportion of variance explained**.
- $R^2 = 1 \Rightarrow$ perfect fit ($\text{RSS} = 0$)
- $R^2 = 0 \Rightarrow$ model no better than predicting $\bar{y}$
- $R^2 < 0 \Rightarrow$ model is *worse* than a constant (can happen in cross-validation)



Key: $\text{TSS} = \text{RSS} + \text{explained}$.

$$R^2 = \frac{\text{explained}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}}.$$

MAE, MSE, RMSE, MAPE

Metric	Formula	Key property
MAE	$\frac{1}{n} \sum y_i - \hat{y}_i $	Same unit as y . Robust: large errors not magnified
MSE	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Same unit as y^2 . Penalises large errors strongly; differentiable
RMSE	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$	Same unit as y . Most common for reporting
MAPE	$\frac{1}{n} \sum \frac{ y_i - \hat{y}_i }{\max(\varepsilon, y_i)}$	Unit-free: can compare across different tasks

MAE: robust, same units

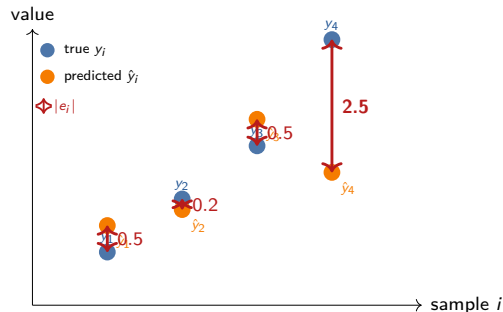
MSE: outlier-sensitive

MAPE: unit-free comparison

Visualising the metrics on a simple example

Suppose we have **4 predictions** (sorted by $\hat{y}$):

i	y_i	$\hat{y}_i$	$e_i = y_i - \hat{y}_i$
1	1.0	1.5	-0.5
2	2.0	1.8	+0.2
3	3.0	3.5	-0.5
4	5.0	2.5	+2.5 (outlier)



- **MAE** = $\frac{0.5+0.2+0.5+2.5}{4} = 0.93$
- **MSE** = $\frac{0.25+0.04+0.25+6.25}{4} = 1.70$
- **RMSE** = $\sqrt{1.70} = 1.30$
- **MAPE** = $\frac{0.5+0.1+0.17+0.5}{4} = 0.32$

Note: MSE/RMSE is **much larger** relative to MAE because sample 4's error (2.5) is squared.

```
from sklearn.metrics import (
    r2_score,
    mean_absolute_error,
    mean_squared_error,
    root_mean_squared_error,  # sklearn >= 1.4
    mean_absolute_percentage_error
)

r2 = r2_score(y_true, y_pred)
mae = mean_absolute_error(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
rmse = root_mean_squared_error(y_true, y_pred)
mape = mean_absolute_percentage_error(
    y_true, y_pred)

# sklearn < 1.4: no dedicated RMSE function
rmse = mean_squared_error(
    y_true, y_pred, squared=False)
```

When to use which?

- **RMSE**: general purpose; same unit as y
- **MAE**: when large errors should not dominate
- R^2 : model selection (default in sklearn)
- **MAPE**: comparing models across different datasets or units

Linear regression: the simplest model

Idea. The model predicts $\hat{y}$ as a **weighted sum** of the features:

$$\hat{y} = w_0 + w_1x_1 + \cdots + w_px_p$$

Each weight w_j says how much feature j influences the output. w_0 is the intercept (baseline value when all features are zero).

Training. Find the weights that make predictions as close as possible to the true values. We minimize the **sum of squared residuals**:

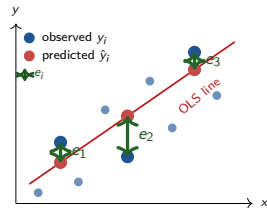
$$\min_w \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Why squared errors?

- Large errors are penalized more strongly.
- Squaring makes the objective smooth and differentiable, so there is an *exact closed-form* solution.

Limitation

When two features are highly correlated (e.g. cholesterol and LDL), OLS may assign very large, unstable weights. The fix is to add a **penalty** on weight magnitude — next slide.



OLS minimises the sum of squared green arrows:

$$\min_w \sum e_i^2.$$

Regularisation: Ridge and Lasso

Problem. When features are many or correlated, OLS tends to give very large weights. A model with huge weights fits training data perfectly but performs poorly on new data — it has *memorised noise*.

Solution. Add a penalty term that discourages large weights:

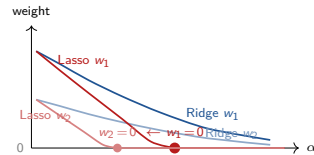
$$\underbrace{\sum (y_i - \hat{y}_i)^2}_{\text{fit well}} + \alpha \underbrace{(\text{penalty on weights})}_{\text{stay small}}$$

α controls the trade-off: $\alpha = 0$ is plain OLS; larger α forces smaller weights.

Two penalties, two behaviors:

Ridge	Lasso
penalty = $\sum_j w_j^2$	penalty = $\sum_j w_j $
All features kept.	Some weights become exactly 0 .
Weights shrink but never hit zero exactly.	$\Rightarrow$ automatic feature selection .

How weights change as α increases:



Ridge: weights shrink smoothly towards 0 but never reach it — all features always used.

Lasso: weights reach exactly 0 and stay there. Once a coefficient is 0, that feature is *removed* from the model. This is why Lasso performs automatic **feature selection**.

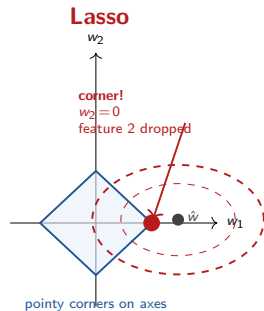
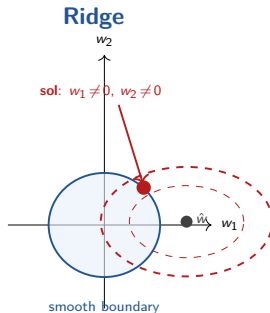
Ridge vs. Lasso: why does Lasso zero out weights?

To understand *why* the penalties behave differently, consider a 2-weight model: w_1 and w_2 .

Red ellipses are “equal-error” contours: every (w_1, w_2) on the same ellipse gives the same training error. Smaller ellipse = lower error. The ellipses shrink toward the centre $\hat{w}$, the unconstrained OLS solution (no penalty).

Blue region is the constraint imposed by the penalty: weights must stay *inside* this region.

The **solution** is where the shrinking ellipse first touches the boundary of the blue region.



The key: shape of the blue region

- **Ridge** → circle (smooth boundary). Ellipses almost never touch an axis $\Rightarrow$ both $w_1 \neq 0$ and $w_2 \neq 0$.
- **Lasso** → diamond (corners on axes). Ellipses very likely touch a corner $\Rightarrow$ one weight = 0 exactly.

Other regression model families

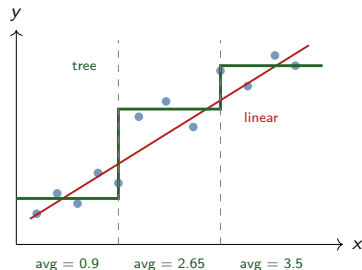
The same learning ideas used for classification can be adapted for regression: instead of predicting a class label, the model predicts a **continuous number**.

Family	Sklearn regressor
k-NN	KNeighborsRegressor
SVM	LinearSVR
Decision Tree	DecisionTreeRegressor
Random Forest	RandomForestRegressor
MLP	MLPRegressor

The **API is identical**: `fit(X, y)` and `predict(X)`. The key difference is that the output of `predict` is a continuous value, and the loss function is a regression loss (e.g. MSE), not a classification loss.

Intuition: Decision Tree for regression

A decision tree splits the feature space into rectangular regions. For regression, it predicts the **average** y of all training points in each region.



Tuning α : what does GridSearchCV do?

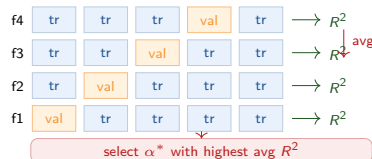
Why can we not just minimise the training loss to find α ? Because $\alpha = 0$ always gives the lowest training loss (plain OLS memorises training data). We need to measure how the model performs on **data it has not seen**.

GridSearchCV procedure (done separately for each candidate α):

- 1 Split data into 10 folds.
- 2 For each fold in turn:
 - Use the **9 other folds** to train Lasso (i.e. find the best weights w^* for this α).
 - Score on the **1 held-out fold**: compute R^2 . This fold was *never used* for training.
- 3 Average the 10 R^2 scores.

Repeat for every α in the grid. Pick $\alpha^* =$ the value with the **highest average score**.

For one candidate α , 4 of the 10 folds shown:



After selecting α^* : re-train Lasso once on *all* data to get the final model.

Final Lasso on all data with α^*

No leakage

The held-out fold is never seen during training or α selection. Same principle as in the nested CV for classification.

Tuning Lasso with GridSearchCV

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV
import numpy as np

X = df[data.feature_names]
y = df['target']

lasso = Lasso(random_state=0, max_iter=10000)
# 30 alpha values on a log scale
alphas = np.logspace(-4, -0.5, 30)

# 10-fold CV, default score = R2
clf = GridSearchCV(lasso,
    [{"alpha": alphas}], cv=10)
clf.fit(X, y)

scores = clf.cv_results_["mean_test_score"]
scores_std = clf.cv_results_["std_test_score"]
print(clf.best_estimator_)
# Lasso(alpha=0.0126, max_iter=10000)
```

Protocol

- **30 candidate** α values on a log scale from 10^{-4} to $10^{-0.5}$
- **10-fold CV** to estimate generalisation R^2 for each α
- Best α returned as `clf.best_estimator_`

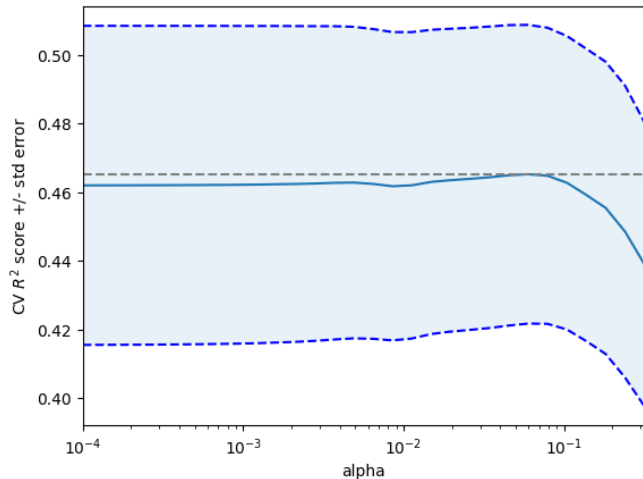
A log scale is used because the effect of α is multiplicative: going from 0.001 to 0.01 has the same proportional impact as going from 0.1 to 1.0.

Lasso CV curve: R^2 vs. α

```
std_error = scores_std / np.sqrt(n_folds)
plt.semilogx(alphas, scores)
plt.semilogx(alphas, scores +
             std_error, "b--")
plt.semilogx(alphas, scores -
             std_error, "b--")
plt.fill_between(alphas,
                scores + std_error,
                scores - std_error, alpha=0.1)
plt.axhline(np.max(scores),
            linestyle="--", color=".5")
```

- **Low** α : model keeps many large weights, may overfit
- **Optimal** α : maximises CV R^2
- **High** α : too sparse, underfits

The shaded band shows $\pm$ std error over folds.
The grey dashed line marks the maximum R^2 .



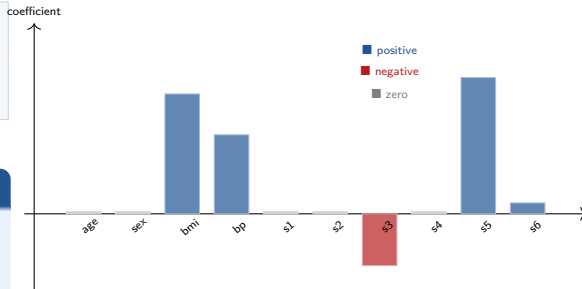
Best Lasso estimator and sparse coefficients

```
clf.best_estimator_.coef_  
# array([ 0. , -0. , 441. , 288. ,  
#        -0. ,  0. , -185. ,  0. ,  
#        504. ,  44. ])
```

What Lasso selected

Several coefficients are **exactly zero**: the model automatically removed those features. Only `bmi`, `bp`, `s3`, `s5`, `s6` contribute.

This is the **feature selection** property of Lasso. Ridge would keep all 10 features with small but non-zero weights.



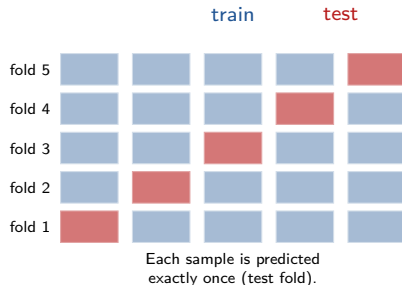
Cross-validated predictions: honest evaluation

```
from sklearn.model_selection import  
    cross_val_predict  
  
y_pred = cross_val_predict(  
    clf.best_estimator_, X, y, cv=10)
```

Why not just call `model.predict(X)`?

If we fit a model on all of X and then predict on the same X , we get **training error** — optimistic and misleading.

`cross_val_predict` ensures that each sample's prediction is made by a model that has **never seen** that sample. This gives an honest estimate of generalisation error.

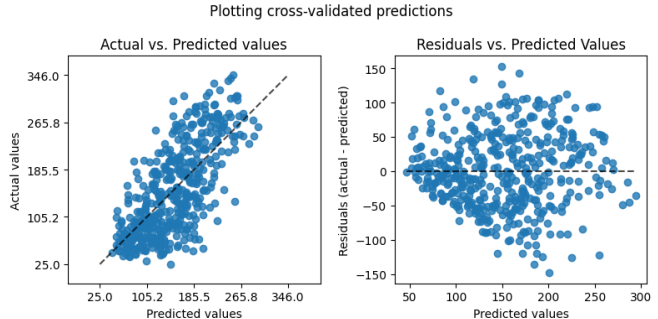


PredictionErrorDisplay

```
from sklearn.metrics import
    PredictionErrorDisplay
fig, axs = plt.subplots(ncols=2,
                        figsize=(8,
                                4))

PredictionErrorDisplay.
    from_predictions(
        y, y_pred=y_pred,
        kind="actual_vs_predicted",
        ax=axs[0])
PredictionErrorDisplay.
    from_predictions(
        y, y_pred=y_pred,
        kind="residual_vs_predicted",
        ax=axs[1])
```

- **Left:** actual vs. predicted. Points on the diagonal = perfect predictions. Spread around diagonal = typical error.
- **Right:** residual $e_i = y_i - \hat{y}_i$ vs. $\hat{y}_i$. Centered at 0 with no pattern. Patterns say that the model is missing something.



Left: each point's horizontal distance from the diagonal equals its residual. Right: the same residuals are plotted explicitly on the y-axis. The two plots show the same errors from different angles.

Reading the prediction error plots

Actual vs. predicted (left plot)

- Points **on the diagonal**: perfect prediction
- Symmetric spread around diagonal: acceptable noise
- **Systematic curve**: the true relationship is non-linear; a linear model is not enough
- **Funnel shape**: errors grow with $\hat{y}$ (heteroscedasticity)

Residuals vs. predicted (right plot)

- Residuals should be **centred around zero**
- **Random scatter**: good model specification
- **U-shape or trend**: missing non-linear terms
- **Funnel**: variance of errors depends on $\hat{y}$
- Large isolated residuals: potential outliers

diagonal = perfect

random residuals = good

patterns = missing structure

Model comparison: nested cross-validation

We compare LR, Decision Tree (DT), Random Forest (RF). The goal is to estimate the true generalisation error of each **model family** without data leakage.

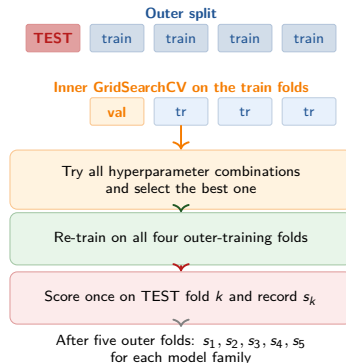
Outer loop (for fold $k = 1 \dots 5$):

- 1 Keep fold k as the **test fold**.
- 2 On the remaining folds, run an **inner GridSearchCV**.
- 3 Select the best hyperparameters, re-train on the outer-training folds.
- 4 Evaluate once on the test fold and record s_k .

Key guarantee

The test fold never influences hyperparameter selection or training. The scores $s_1, \dots, s_5$ can be used to compare model families fairly.

Structure of the loop (outer fold $k = 1$):



Reading the results and choosing the best model

After nested CV, each model has **5 R^2 scores** (one per outer fold). These are not averages — they are *paired observations* on the same data splits.

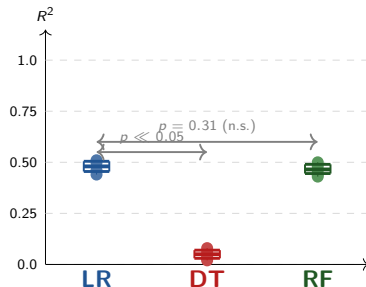
	s_1	s_2	s_3	s_4	s_5	mean
LR	.44	.51	.48	.46	.50	.48
DT	.02	.08	.04	.06	.03	.05
RF	.43	.50	.46	.44	.48	.46

Can we just pick the highest mean? Not quite. LR and RF have similar means. We need a **statistical test** to decide.

Wilcoxon signed-rank test

Compare two models on the *same 5 folds*. For each fold: $d_k = s_k^{\text{LR}} - s_k^{\text{RF}}$. Test H_0 : the median of $d_k = 0$. If $p < 0.05$, the difference is significant; otherwise, prefer the **simpler** model (LR).

Visualise the 5 paired scores per model:



DT is clearly worse (non-overlapping distributions). LR vs. RF: not significantly different $\Rightarrow$ **prefer LR** (simpler, more interpretable).

The full pipeline: from comparison to deployment

Nested CV tells you **which model family** generalises best. It does not give you the final model to deploy.

1. Model selection:

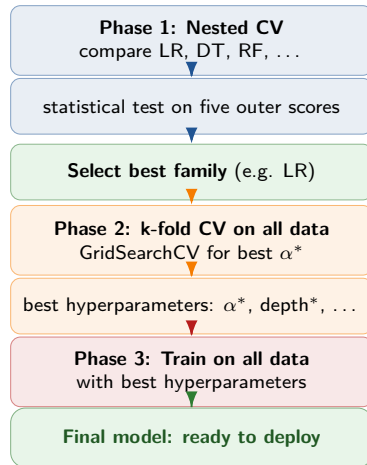
- Run nested CV for each family (LR, DT, RF, ...).
- Compare the five outer-fold scores with a paired statistical test.
- Select the best family, e.g. LR.

2. Hyperparameter selection:

- Run GridSearchCV on the whole dataset for the selected family.
- Find the final value, e.g. α^* for Lasso.

3. Final training:

- Re-train on all available data with the selected hyperparameters.
- Deploy this final model.



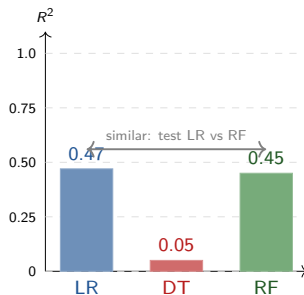
Typical results: LR vs. DT vs. RF

After nested CV, each model has 5 outer-fold scores. Mean values (approx.):

Model	$\bar{R}^2$	RMSE	MAPE
Linear Reg.	0.47	55	0.37
Decision Tree	0.05	80	0.51
Random Forest	0.45	57	0.38

Observations:

- **DT is clearly worse:** low R^2 , high RMSE. It overfits the training data; its outer-fold scores are much lower than its training score.
- **LR and RF are similar.** With $n = 442$ and mostly linear relationships, the simpler model (LR) matches the ensemble.
- Use a **statistical test** (Wilcoxon) to confirm LR vs. RF: if $p > 0.05$, prefer LR (Occam's razor).



Bias–variance trade-off

Every model makes two kinds of error:

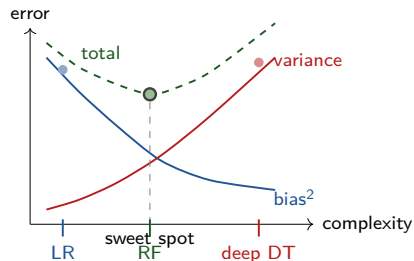
Bias = *wrong by design*. A linear model on a curved signal will always miss the pattern, no matter how much data you add. It is systematically too simple.

Variance = *too sensitive to the data*. A deep decision tree memorises every training point. Give it slightly different data and its predictions change a lot. It overfits.

Total error $\approx \text{bias}^2 + \text{variance} + \text{noise}$.

The goal is to find a model in the **sweet spot**: complex enough to capture the signal (low bias), but not so complex that it memorises noise (low variance).

On a small dataset ($n = 442$) there is not enough data to reliably estimate complex patterns. A simple model like Linear Regression often matches or beats Random Forest.



In our experiment: LR sits on the left (simple, some bias, low variance). Deep DT on the right (complex, low bias, high variance). RF is in between: it averages many trees to reduce variance.

Conclusions

Core ideas

- Regression predicts a **continuous target**.
- Choose the metric to match the application: RMSE for general use, MAE for robustness, MAPE to compare across datasets.
- R^2 measures the proportion of variance explained by the model.

Methodological message

- Lasso performs **automatic feature selection** via ℓ_1 penalty.
- Tune α with GridSearchCV; evaluate with cross-validated predictions.
- Use **residual plots** to diagnose model problems.
- Apply **nested CV** to avoid leaking test information into tuning.
- On small datasets, simple models often win (bias–variance trade-off).

choose the right metric

regularise to generalise

never leak the future

Questions?